Dynamic Limit Adjustment Using UltraEdge and AMP

Reference : RA 447

Description

This demo demonstrates how a test program can interact with an **UltraEdge (UE)** system by uploading a **Docker container**.

The container includes:

- An **AMP DAS (Data Acquisition System)**
- A mechanism to compute a **new low limit** for a specific test

Once computed, this limit is transmitted via a **FIFO (First-In-First-Out)** queue to the test program, allowing it to update the test limit dynamically in real time.

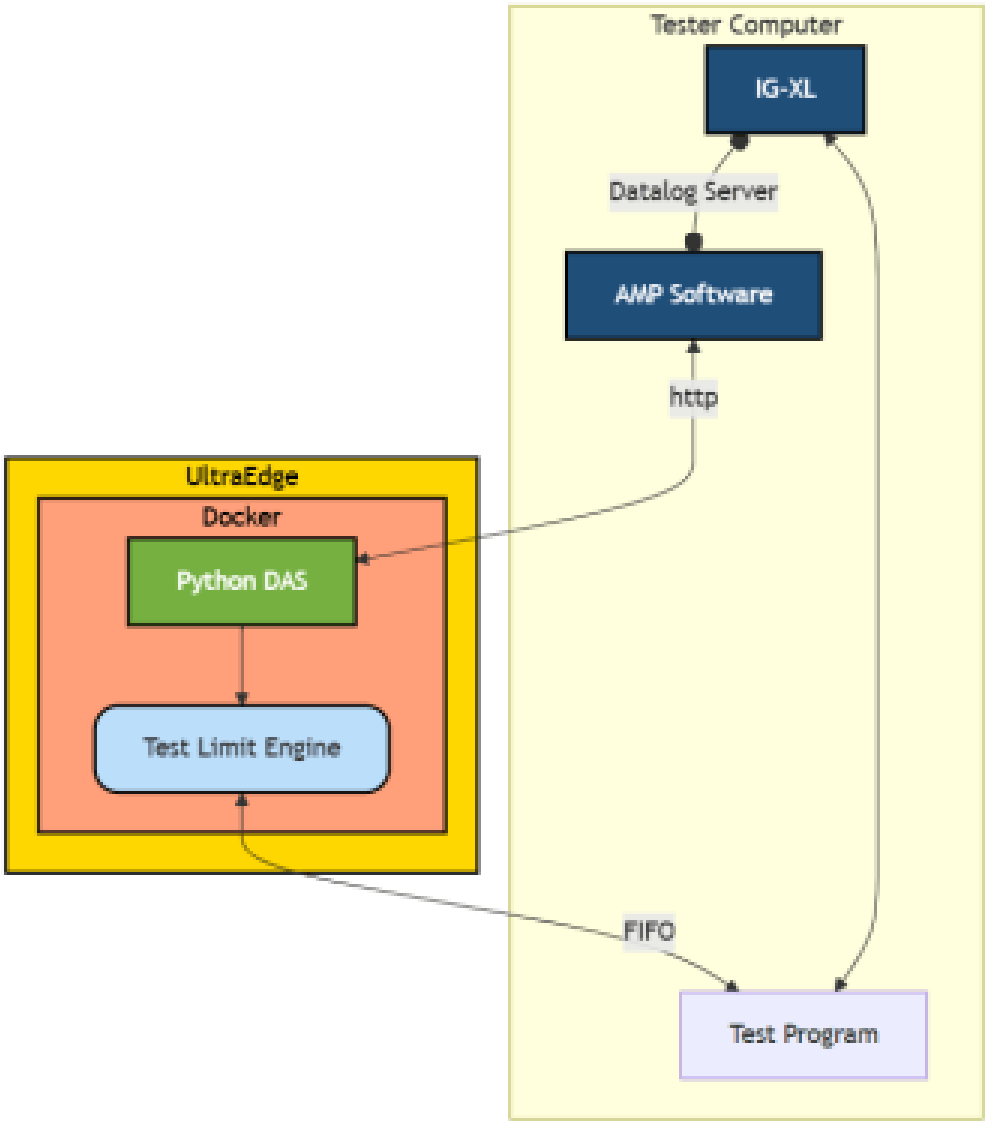
For demonstration purposes, the new test limit is calculated as the **average of all previous test results**. As the test program continues, the limit is automatically updated, showcasing the system's adaptive behavior.

The primary objective of this demo is to highlight the **ecosystem synergy between AMP and UltraEdge**. By combining AMP's analytics with UE's execution capability, the test program becomes smarter and evolves based on collected data.

Key Highlights

- **UltraEdge and AMP Integration**
  Demonstrates interaction between AMP and UE to enable smarter test behavior.
- **Dynamic Test Limits**
  Real-time limit updates based on live results.
- **Adaptive Test Program**
  Test conditions evolve based on previous outcomes.

Visual Summary



Source Code

> **NOTE**
>
> The source code will be available in a future release via a dedicated Git repository.

General Information

- **Language**: Python
- **Platform**: IG-XL 10.50.00
- **System**: UltraEdge with AMP

Advanced Details - RA 0450

Understanding the Dockerfile

In this section, we dive into the core components of the Docker container used in RA 0450. The container hosts a Python-based DAS (Data Acquisition System) service running on an UltraEdge system. Below, we analyze key instructions in the `Dockerfile`.

> **TIP**
>
> The full `Dockerfile` can be found here: source/Docker/Dockerfile

Key Dockerfile Commands Explained

`RUN pip install flask`

This line installs **Flask**, a lightweight web framework for Python. Flask is widely used to build RESTful APIs, which makes it a natural fit for serving communication endpoints in a microservice environment like this DAS.

In this use case, Flask enables the DAS inside the Docker container to:

- Act as a **local web server** on the UltraEdge
- Respond to incoming HTTP requests from the tester
- Serve dynamically computed data such as updated test limits

Flask's simplicity and minimal footprint make it ideal for containerized microservices.

`EXPOSE 5001`

This command declares **port 5001** as the port on which the container will listen. It is essential to enable communication between the:

- **Tester Host** (running IGXL or MST)
- **Docker Container** (running on the UltraEdge)

By convention in this setup, port `5001` is used to:

- Receive incoming requests from the test program
- Respond with computed data from the DAS (e.g., updated test limits)

> **NOTE**
>
> While `EXPOSE` does not publish the port to the host by itself, it **documents the intention** and allows tools like Docker Compose or manual publishing (`-p 5001:5001`) to expose it.

In summary, these two lines are essential for enabling the Python DAS application to:

- Be reachable from outside the container (via port 5001)
- Serve test results or configuration updates using a lightweight Flask server

The combination of Flask + port exposure establishes the communication bridge between your test program and the DAS logic running in an isolated UltraEdge container.

Understanding the Python DAS (`pydas.py`)

The Python DAS is a Flask-based web service that:

- Receives AMP messages via HTTP POST requests
- Extracts test data from incoming JSON payloads
- Computes the new **low limit** as the average of recent results
- Sends this value back to the test program via a FIFO file descriptor

Architecture Overview

- **Communication with Tester**: The tester sends AMP messages (like `TEST_DATA`, `TEST_END`, etc.) to the DAS.
- **Threading**: Test data is managed in parallel threads to avoid blocking.
- **Data Storage**: Test values are stored in a list of `TestData` instances.
- **Computation**: On `TEST_END`, the average of the `RESULT` values is computed.
- **Result Reporting**: The final result is sent through a FIFO to the tester program.

Key Components Explained

- `TestData` class: Used to encapsulate one test's data (test number, site, result, and limits).
- Flask routes: Handle various AMP message types:
  - `/SUBLOT_START`: Initializes the process
  - `/TEST_START`: Clears the previous data
  - `/TEST_DATA`: Parses and stores new data
  - `/TEST_END`: Triggers computation and sends results

- `/SUBLOT_END`: Final clean-up

Inter-Process Communication

The line:

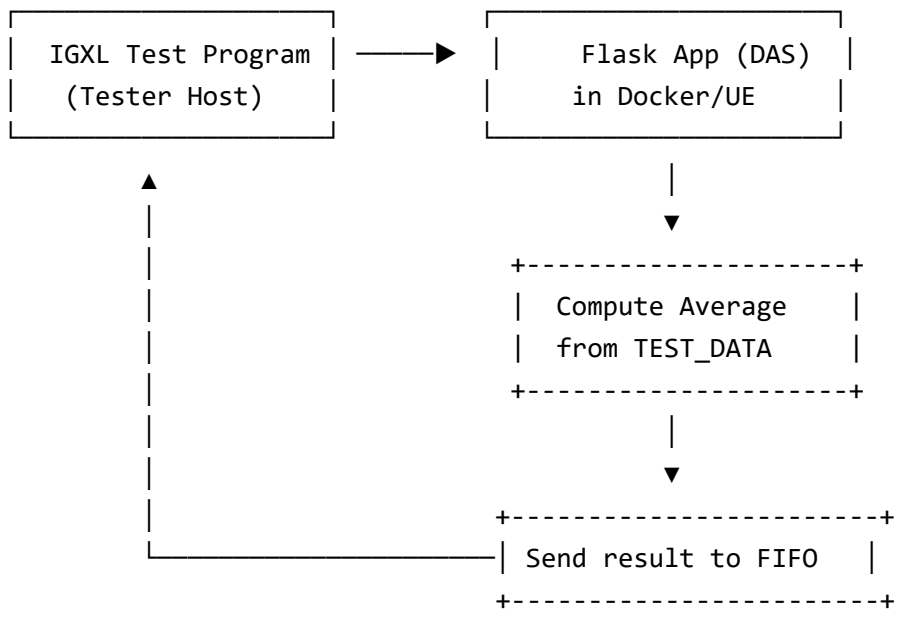
```
fifo_output = "/app/exec/fifo_to_TP"
output_stream = open(fifo_output, 'w')
```

is critical for communication between the Docker-based DAS and the IGXL tester. It uses a **named FIFO pipe** to push results back into the tester host.

Example result message sent:

```
CODE:0.87|report:[S:0,TN:1234,R:0.5,LLM:-1,HLM:1],...
```

Message Flow Diagram



This illustrates how the test program interacts with the DAS inside UltraEdge:

1. Test program sends messages (e.g., `TEST_DATA`, `TEST_END`) to Flask routes.
2. Flask handles and processes data.
3. Result is computed.
4. Result is sent back to the tester using FIFO.

Test Program & Tester Host Communication

On the tester host, the IGXL test program drives interaction with the UltraEdge system. It performs the following steps:

Step 1: Connect to UltraEdge

In the `OnProgramLoaded` event:

```
UE.Connect "10.100.100.1", 2023
```

This establishes a TCP connection with the UltraEdge server (port 2023).

Step 2: Start Session and Launch Docker

In the `OnProgramValidated` event:

```
UE.SecureSession.Start ""
UEToolsDocker.SetupCommunicationPath("pydas")
UEToolsDocker.UE_InstallandStartDockerApp
```

This:

- Starts a secure session
- Creates FIFO for tester-to-UE communication
- Installs and runs the Dockerized DAS

Supporting Module: `UEToolsDocker`

This module includes utilities to:

- Transfer the `pydas.tar.gz` archive to UltraEdge
- Load the Docker image
- Start the container mapped on port 5001
- Send/receive data via FIFO

Key method:

```
UE.Application("pydas").RunDockerContainer("pydas", "/app/pydas.py", "pydas", "5001:5001")
```

This runs the Flask app inside Docker and exposes it to tester.

Reading the Results

```
UE.ReadString ApplicationName & FROMUE_FIFO, data
```

This reads the output from the FIFO and extracts:

- `CODE:<limit>`: the new test limit
- `report:<raw>`: a report string of test results

Test Function

```
Call TheExec.Flow.TestLimit(measure, lowlimit, 10, , , , unitVolt, , , , , , , , , , 123)
```

The test program uses the updated low limit received from the DAS to dynamically adjust pass/fail criteria.

Next section: How the AMP JSON messages are structured and parsed in Python.